

# FILESYSTEM

## Introducción a los Sistemas de Archivos

(Basado en Tanenbaum, Carrier, Microsoft, Stevens y Arpaci-Dusseau)

Un sistema de archivos es el componente esencial de un sistema operativo que traduce la complejidad del almacenamiento físico en una abstracción lógica y manejable para usuarios y aplicaciones. Su función primordial es garantizar que los datos no solo se almacenen de manera eficiente, sino que también sean accesibles, seguros y persistentes. A diferencia de la visión simplista de archivos como meros "contenedores de bytes", un sistema de archivos moderno es una estructura jerárquica y altamente optimizada que integra mecanismos de organización, control de acceso, recuperación ante fallos y adaptación a tecnologías emergentes.



### Abstracción del almacenamiento físico:

Uno de los principales objetivos de un sistema de archivos es ocultar la complejidad del almacenamiento físico, presentando una vista lógica para el usuario. El sistema traduce bloques físicos, como sectores de disco, en estructuras lógicas como archivos y directorios que los usuarios pueden manipular fácilmente.

- En UNIX, un archivo como `/home/usuario/informe.pdf` oculta los detalles de cómo se almacenan sus 100 bloques de 4 KB en el disco.
- En FAT32, un archivo se almacena como una cadena de clústeres enlazados (bloques de 4 KB), gestionados por la tabla FAT.

### Metadatos y Organización Física

La arquitectura de un sistema de archivos se sustenta en dos pilares: *metadatos* y *organización física*. Los metadatos, como los inodos en UNIX o las entradas de la Master File Table (MFT) en NTFS, almacenan información no solo sobre la ubicación de los bloques de datos, sino también sobre permisos, timestamps y atributos extendidos. Estos últimos son clave en análisis forenses, donde detalles como el "tiempo de creación" (MACB) pueden reconstruir actividades sospechosas. Por otro lado, la organización física determina cómo se asignan los bloques en el disco. Mientras FAT32 emplea una tabla de clústeres enlazados propensa a fragmentación, sistemas como ext4 utilizan asignación dinámica con punteros directos e indirectos, reduciendo la fragmentación y permitiendo archivos de hasta 16 terabytes.

### Persistencia y Recuperación ante Fallos

La persistencia de datos es otro eje fundamental. Técnicas como el *journaling* (ext3/4) o el *copy-on-write* (ZFS, Btrfs) aseguran que, incluso ante fallos de energía o corrupción, el sistema pueda recuperar un estado consistente. Por ejemplo, el journaling registra las

operaciones pendientes en un "diario" antes de aplicarlas al disco, mientras que el copy-on-write evita la sobrescritura directa, creando copias modificadas de los bloques afectados.

## Seguridad y Control de Acceso

En cuanto a seguridad, los sistemas de archivos modernos van más allá de los permisos clásicos de UNIX (rwx). NTFS implementa listas de control de acceso (ACLs) para definir permisos granulares por usuario o grupo, mientras que ZFS integra cifrado a nivel de bloque y verificación de integridad mediante hashes SHA-256. Estas características no solo protegen contra accesos no autorizados, sino también contra la corrupción silenciosa de datos, un riesgo crítico en entornos empresariales.

Finalmente, la eficiencia se logra mediante estrategias como la *deduplicación* (ZFS), que elimina bloques idénticos para ahorrar espacio, o el *caching* a nivel de kernel, que almacena bloques frecuentemente accedidos en memoria RAM. Estas optimizaciones son vitales en aplicaciones de alto rendimiento, como bases de datos o sistemas de análisis en tiempo real

## Fundamentos teóricos

### 1. Métodos de organización de datos

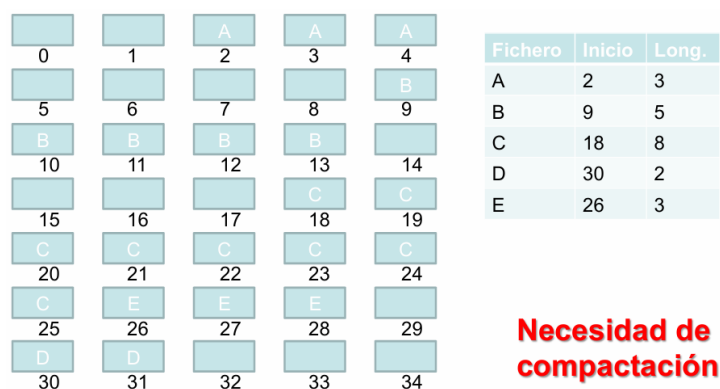
La organización de ficheros en un sistema de archivos determina cómo se almacenan, gestionan y acceden los datos en un dispositivo de almacenamiento. La forma en que los archivos son organizados afecta directamente el rendimiento, la eficiencia de acceso y la gestión del espacio de almacenamiento. Existen tres estrategias principales de organización de ficheros:

- Organización Contigua
- Organización Enlazada
- Organización Indexada
- Organización Basada en Árboles Equilibrados

#### 1.1. Organización Contigua

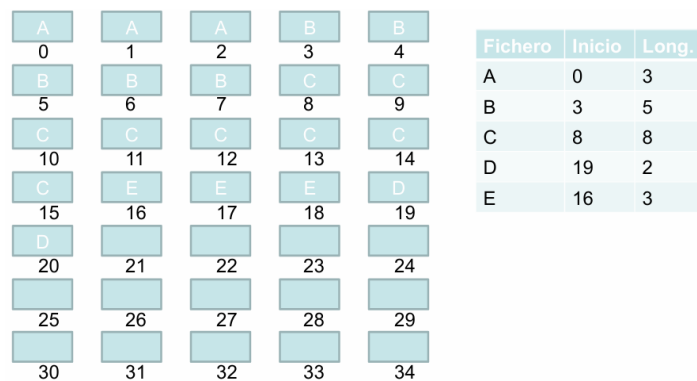
La organización contigua consiste en almacenar todos los bloques de un archivo de manera contigua en el disco, es decir, uno a continuación del otro.

Por ejemplo; este método es común en sistemas de archivos ISO-9660, utilizado en CD-ROM, y cintas magnéticas donde los datos puedan leerse secuencialmente de manera eficiente



En este esquema, el sistema operativo asigna un rango de bloques contiguos al archivo cuando se crea. Si el archivo crece más allá de su tamaño inicial, puede ser necesario moverlo a un área más grande del disco para mantener la contigüidad, lo que puede implicar una operación costosa de reorganización de datos.

### Asignación contigua (compactación)



#### Ventajas

- El acceso secuencial es óptimo en medios lineales como cintas magnéticas, donde el cabezal no necesita moverse. Además, el sistema operativo puede precargar múltiples bloques en memoria con una sola operación de E/S, reduciendo la latencia.
- Optimización de lectura secuencial; ideal para archivos multimedia (vídeos, audio) o grandes volúmenes de datos que se leen secuencialmente.

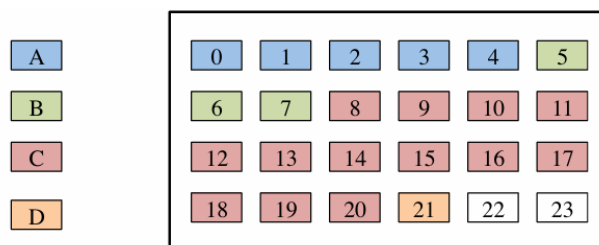
#### Desventajas

- La fragmentación externa es un problema crítico en discos reutilizables. Por ejemplo, un disco con 10 GB libres divididos en fragmentos de 1 GB no puede almacenar un archivo de 5 GB sin desfragmentación
- Para reducir la fragmentación externa, el sistema debe reorganizar los archivos mediante un proceso de compactación, que es costoso en términos de tiempo y recursos

En cuanto a *seguridad*, en sistemas FAT12, la organización contigua no proporciona mecanismos avanzados de control de acceso o cifrado, lo que podría exponer los datos a accesos no autorizados.

*Estado Actual:* Este enfoque se considera obsoleto para discos reescribibles, pero sigue siendo útil en medios de solo lectura como DVDs y Blu-rays y sistemas embebidos simples.

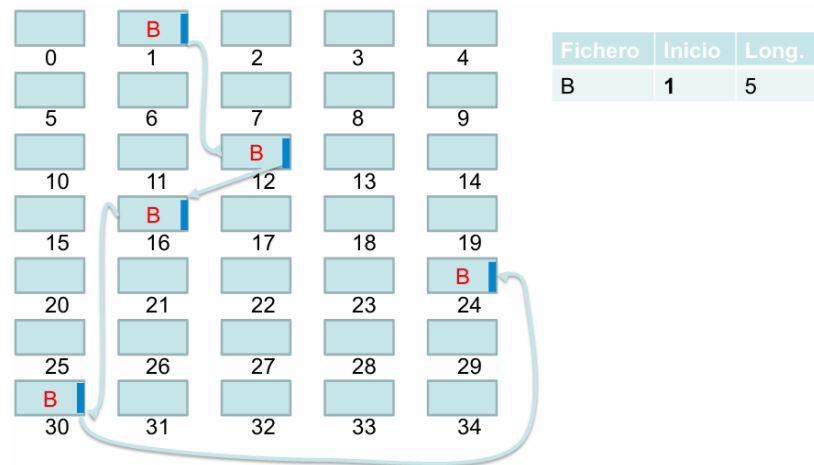
#### Ejemplo de Organización contigua: CD-ROM(ISO-9660)



### 1.2. Organización Enlazada (o Encadenada)

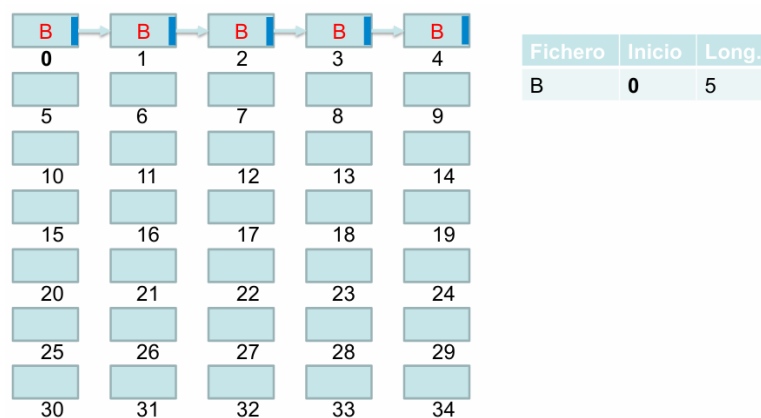
La organización enlazada (también conocida como encadenada) almacena los bloques de un archivo en cualquier lugar disponible en el disco, pero cada bloque contiene un puntero al siguiente bloque del archivo. Cada bloque contiene un puntero al siguiente, permitiendo almacenamiento no contiguo. Es la base de sistemas como FAT32.

Este método se emplea en sistemas como FAT (File Allocation Table). En FAT32 (dispositivos USB), un archivo de 10 MB se divide en clústeres de 32 KB (320 clústeres). La tabla FAT almacena las cadenas de clústeres, pero acceder al bloque 150 requiere recorrer 149 entradas, lo que introduce latencia ( $O(n)$ ).



En lugar de depender de la contigüidad física de los bloques, la estructura enlazada permite que los bloques de un archivo estén dispersos en el disco. El sistema de archivos mantiene una tabla de asignación (como FAT) o un puntero dentro de cada bloque para rastrear la cadena de bloques que forman el archivo.

#### Asignación encadenada (consolidación)



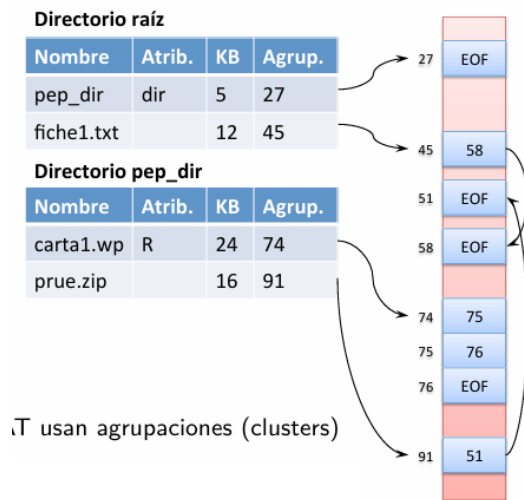
#### Ventajas

- No produce fragmentación externa, ya que los bloques dispersos pueden reutilizarse fácilmente, lo que evita problemas de fragmentación externa.
- La gestión es simple: Agregar o eliminar bloques es sencillo, ya que solo implica actualizar los punteros en la tabla FAT.

#### Desventajas

- Acceso aleatorio es ineficiente; para acceder a un bloque específico, es necesario recorrer la cadena de punteros desde el inicio.
- Reduce la capacidad de almacenamiento; cada bloque necesita espacio adicional para almacenar los punteros. Por ejemplo, en un disco de 1 TB con clústeres de 32 KB, la tabla FAT ocupa ~12.5 MB (4 bytes por clúster)

### Ejemplo de Organización enlazada: MS-DOS(FAT)



Las diferentes variantes de FAT (File Allocation Table) utilizan **clusters** en lugar de bloques individuales para asignar espacio en disco a los archivos.

**Un cluster es un conjunto de bloques contiguos.** Esto significa que los bloques que forman un cluster están físicamente adyacentes en el disco.

**Cada archivo ocupa al menos una agrupación en disco.** Si un archivo es pequeño y no llena completamente un cluster, se asigna todo el cluster al archivo, incluso si hay espacio libre dentro del cluster

En cuanto a *seguridad*, FAT32 solo soporta atributos básicos de lectura/escritura, sin ACLs o cifrado. Esto lo hace vulnerable en entornos donde se requiere control granular.

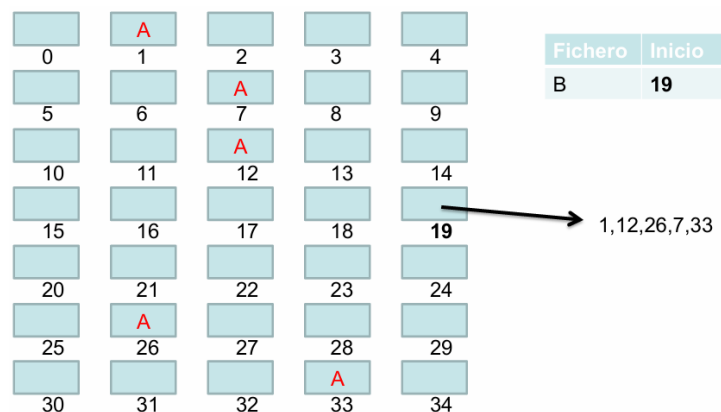
*Estado Actual:* Obsoleto para sistemas críticos. Persiste en dispositivos USB por compatibilidad, pero es reemplazado por **exFAT** en almacenamiento moderno.

### 1.3. Organización Indexada

En la organización indexada, el sistema de archivos mantiene una estructura de índice que apunta directamente a los bloques de datos que componen un archivo. Esto permite un acceso rápido y eficiente tanto para lectura secuencial como aleatoria.

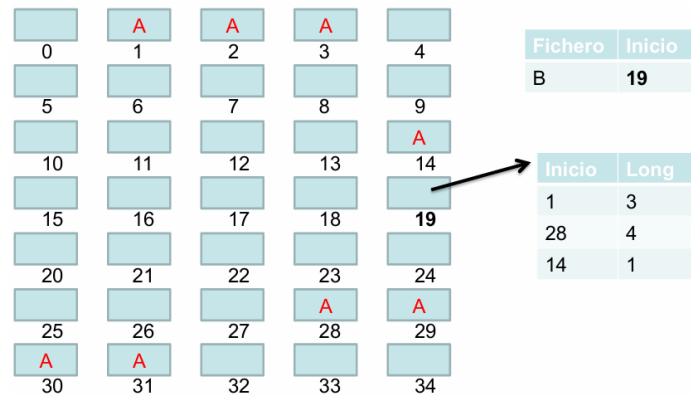
Este esquema es utilizado en sistemas como UNIX (ext2/ext4) y NTFS de Windows, los archivos se gestionan mediante estructuras como los inodos y la Master File Table (MFT), respectivamente. Los inodos almacenan punteros directos e indirectos a los bloques de datos del archivo, mientras que la MFT en NTFS almacena los metadatos y las ubicaciones físicas de los archivos

#### Asignación indexada por bloques



Este diseño permite una jerarquía de índices que facilita la gestión de archivos grandes mediante un sistema multinivel de referencia

### Asignación por porciones



### Ventajas

- Acceso aleatorio eficiente; el acceso directo mediante índices permite localizar cualquier bloque rápidamente.
- La estructura indexada permite recorrer los bloques en orden, facilitando el acceso secuencial.
- Los índices multinivel permiten manejar archivos de gran tamaño

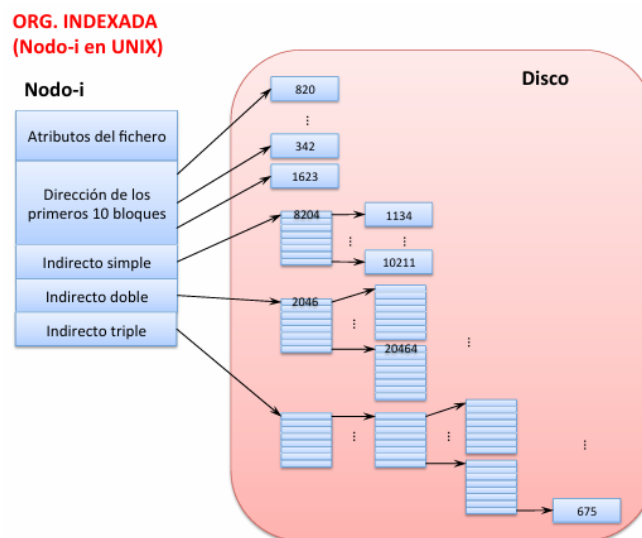
### Desventajas

- La fragmentación interna es común: un archivo de 100 bytes en un bloque de 4 KB desperdicia 3.9 KB.
- Complejidad en la creación y actualización de índices, ya que requiere algoritmos complejos, como *extents*.

En cuanto a *seguridad*, en sistemas ext4 se utilizan los Permisos POSIX (rwx) y ACLs opcionales (setfacl -m u:admin:rwx). En cambio en sistemas NTFS utilizan ACLs granulares y cifrado EFS (Encrypting File System) para protección de datos.

*Estado Actual:* Vigente. NTFS y ext4 son estándares en Windows y Linux, respectivamente.

### Ejemplo de Organización indexada: Nodos-I (UNIX)



## 1.4. Organización Basada en Árboles Equilibrados

En esta estrategia, los archivos y sus bloques se almacenan y gestionan mediante una estructura de árbol equilibrado (como un B-Tree o un B+ Tree). El árbol proporciona un mecanismo eficiente para almacenar, buscar y recuperar datos de manera ordenada, lo que mejora el rendimiento en operaciones de acceso aleatorio y secuencial.

Los sistemas de archivos modernos como NTFS, JFS (Journaled File System), ReiserFS y XFS utilizan estructuras de árboles equilibrados para gestionar archivos y directorios. En NTFS, un directorio con 1 millón de archivos se indexa en un árbol B+ balanceado, permitiendo búsquedas en  $O(\log n)$ . Cada nodo interno almacena hashes de nombres, y las hojas contienen entradas de la MFT.

### Ventajas

- La estructura de árbol permite realizar búsquedas rápidas mediante operaciones de búsqueda binaria.
- Rendimiento consistente incluso con petabytes de datos.
- Ideal para servidores de bases de datos o sistemas distribuidos como ZFS, que integra cifrado AES-256.

### Desventajas

- Mayor complejidad en la gestión y reorganización automática de los árboles, ya que requiere algoritmos más complejos.
- Los árboles equilibrados requieren un uso adicional de memoria para almacenar índices, lo que puede resultar en un uso más elevado de recursos.

En cuanto a *seguridad*, ZFS integra cifrado AES-256 con una gestión de claves unificada, proporcionando un alto nivel de seguridad para los datos. En cambio, NTFS, ofrece cifrado EFS y ACLs detalladas para un control preciso de los accesos.

*Estado Actual:* Vigente y en desarrollo: NTFS, APFS (Apple) y ZFS son ejemplos de sistemas de archivos modernos que utilizan árboles equilibrados para garantizar rendimiento y escalabilidad.

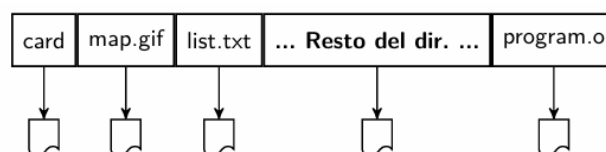
## 2. Estructuras de directorios

Los directorios son archivos especiales que mapean nombres lógicos a identificadores físicos. Su diseño afecta la eficiencia de búsqueda, gestión de permisos y escalabilidad.

### 2.1. Directorio de único nivel

En un sistema de archivos con un directorio de único nivel, todos los archivos están ubicados en un solo directorio. Esta estructura es la más sencilla, pero limita la organización y gestión de archivos, especialmente cuando el número de archivos crece.

Este tipo de estructura utilizaban los sistemas operativos en los años 70. Utilizado en sistemas FAT12 en disquetes y sistemas DOS antiguos y CP/M



**Ventajas:** Tienen implementación directa, ya que no hay necesidad de jerarquías o niveles de directorios. Es simple de implementar, lo que lo hace ideal para sistemas con pocos archivos

**Desventajas:** Al no haber subdirectorios, todos los archivos deben tener nombres únicos, lo que puede generar conflictos si el número de archivos crece. Además, la falta de una jerarquía y categorización de los archivos dificulta la organización eficiente.

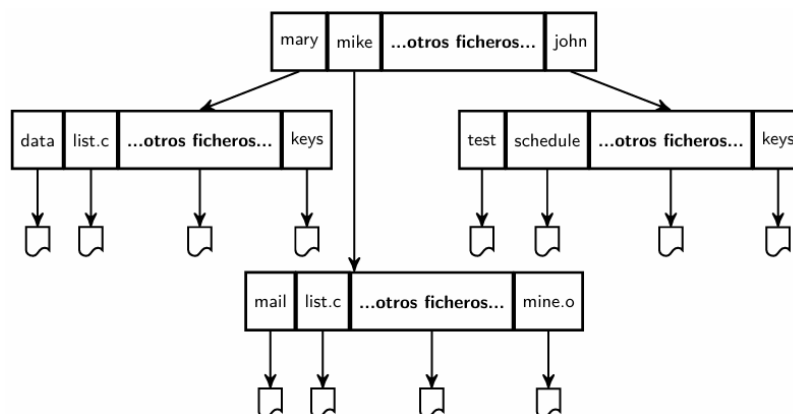
En cuanto a *seguridad*, este tipo de sistema tiene baja protección de archivos, todos los archivos están en un solo directorio, lo que facilita el acceso no autorizado. En cuanto a los permisos, sin controles, ya que no es posible gestionar permisos a nivel de subdirectorios.

*Estado actual:* Este modelo es obsoleto y solo se encuentra en sistemas antiguos o de baja complejidad

## 2.2. Directorio de dos niveles

El sistema de directorio de dos niveles agrega una capa de organización, donde cada usuario tiene su propio directorio. Esto ayuda a reducir la probabilidad de conflictos de nombres, ya que los archivos se agrupan en directorios separados por usuario.

Este sistema se utiliza comúnmente en sistemas operativos multiusuario, como UNIX/Linux, donde cada usuario tiene su propio "home directory". También es implementado en Windows, donde los usuarios tienen carpetas específicas para sus archivos



**Ventajas:** Este sistema mejora la organización al separar los archivos por usuario, evitando la colisión de nombres. Además, proporciona mejor seguridad y privacidad, ya que los archivos de cada usuario están aislados, mejorando la seguridad de los datos. También facilita la búsqueda, ya que la separación por usuario permite una navegación más clara

**Desventajas:** Presenta una eficiencia limitada en la agrupación de archivos relacionados que pertenecen a diferentes usuarios. Además, puede generar cierta complejidad en la navegación, ya que los usuarios pueden tener que cambiar constantemente entre directorios, lo que puede resultar incómodo.

En cuanto a *seguridad*, este sistema proporciona un aislamiento básico de usuarios, ya que los archivos están separados en directorios personales, lo que mejora la seguridad. También permite controlar el acceso a nivel de usuario, aunque sigue siendo limitado en comparación con estructuras más avanzadas. Sigue siendo utilizado en sistemas operativos multiusuario modernos como Linux, Windows y macOS.

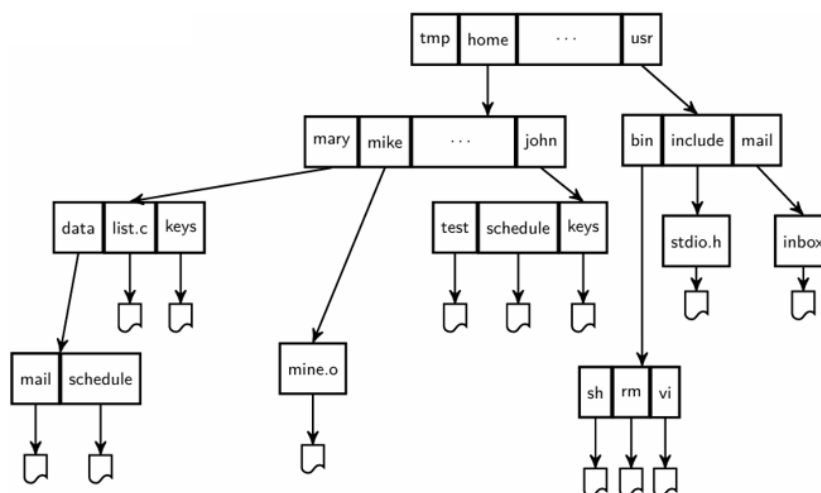


*Estado actual:* Sigue siendo utilizado en sistemas operativos multiusuario modernos como Linux, Windows y macOS.

### 2.3. Directorio con estructura de árbol

Una estructura de árbol permite que un directorio contenga tanto archivos como subdirectorios, creando una jerarquía de múltiples niveles. El directorio raíz actúa como punto de partida desde donde se organizan todos los archivos y directorios.

UNIX/Linux utiliza una estructura jerárquica donde el directorio raíz ("/") es el punto de partida para todos los directorios y archivos. Windows también emplea una estructura jerárquica con unidades de disco (C:, D:) que actúan como raíces separadas. macOS, similar a UNIX, utiliza una estructura jerárquica basada en "/".



**Ventajas:** Este sistema permite una organización eficiente, ya que facilita la agrupación lógica de archivos y subdirectorios. También proporciona flexibilidad, permitiendo agregar, mover o eliminar directorios y subdirectorios según sea necesario. La gestión de nombres relativa y absoluta mejora la navegación mediante el uso de rutas relativas y absolutas.

**Desventajas:** Por otro lado, este tipo de organización implica mayor complejidad, ya que requiere mecanismos más sofisticados para gestionar permisos y referencias de archivos. Además, en sistemas con una gran cantidad de subdirectorios y archivos, la profundidad de la estructura puede afectar el rendimiento.

En términos de *seguridad*, permite un control granular de permisos, asignando niveles de acceso en diferentes niveles del árbol. También incluye seguridad basada en herencia, donde los permisos pueden heredarse de directorio a subdirectorio, lo que simplifica la gestión. Sistemas como SELinux añaden políticas MAC (Mandatory Access Control) para auditoría integrada.

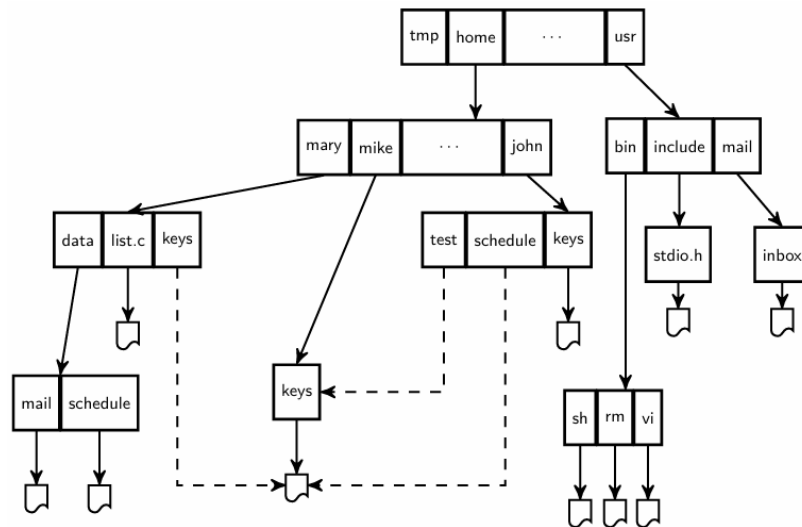
*Estado actual:* Es el modelo más común en sistemas operativos modernos (Linux, Windows, macOS) debido a su flexibilidad y organización eficiente.

### 2.4. Directorio con estructura de grafo acíclico

En un grafo acíclico dirigido (DAG, por sus siglas en inglés), los directorios pueden tener enlaces hacia otros directorios o archivos, permitiendo que un archivo o subdirectorio aparezca en múltiples ubicaciones sin duplicarse.

Este tipo de directorios es utilizado en sistemas de control de versiones como Git, que permite que las ramas apunten a un mismo archivo sin duplicación física. En Unix/Linux se utilizan

enlaces simbólicos (ln -s) para crear referencias a archivos o directorios. NTFS (Windows) soporta enlaces simbólicos y puntos de unión para archivos y carpetas.



**Ventajas:** Este sistema permite una compartición eficiente, ya que los archivos y subdirectorios pueden ser compartidos entre directorios sin necesidad de duplicación, lo que ahorra espacio en el disco. También facilita la creación de alias o enlaces simbólicos, mejorando el acceso a archivos que pueden estar ubicados en varios directorios.

**Desventajas:** Presenta mayor complejidad en la gestión, ya que la existencia de múltiples caminos hacia un archivo puede complicar el mantenimiento del sistema de archivos. También existe el riesgo de inconsistencias; si no se gestiona adecuadamente, la eliminación de archivos compartidos puede llevar a la pérdida de enlaces.

En términos de *seguridad*, permite una gestión avanzada de permisos, proporcionando un control detallado, aunque con la complejidad de manejar múltiples rutas hacia un archivo. Sin una correcta administración, los enlaces simbólicos pueden resultar en configuraciones de seguridad inadecuadas. Herramientas como `find / -type l -xtype f` identifican enlaces rotos o maliciosos.

**Estado actual:** Ampliamente utilizado en sistemas operativos y sistemas de control de versiones debido a la flexibilidad y eficiencia en el uso de espacio.